Queen Mary University of London
School of Electronic Engineering and Computer Science

Lab 5 – Java Threads

Aims & Assumptions

The lab's aim is to understand how to create and start threads in Java, and to observe some of the interference (i.e., race conditions) between threads.

It is assumed that all students have done the following, in advance of this lab:

1. Have spent some time going over (as previously recommended in the **Block 3** topic in QMplus) Oracle's **Java Concurrency tutorial** at

   http://docs.oracle.com/javase/tutorial/essential/concurrency/index.html.

2. Understand the **Blocks 3+4** teaching materials in QMplus for creating and starting threads in Java; in particular, it is expected that students studied the materials included in:
   - all the files (including the .zip file) under the **Block 3** topic folder **L11: Concurrency and Threads (part 1/2)**; and
   - the lecture slides under the **Block 4** topic folder **L12.a: Concurrency and Threads (part 2/2)**.
3. Finally, this also means that students are expected either:
   a) to have followed the installation steps for **OpenJDK 20**, under the **Block 3** topic folder **L11: Concurrency and Threads (part 1/2)**, so they can develop and run Java code in their own laptop;

   OR

   b) to be using one of the computers in the lab, for completing the exercises.

Two Java source code files are provided as "starter code" for this lab; it is recommended that you have a look at that code, in advance of the lab:

- Class `Incrementer` extends `Thread` and is our thread class. Its `run()` method does nothing useful at the moment.
- Class `MainApplication` creates 2 `Incrementer` threads; this class also does nothing else useful yet …

Task 1: Implement `Runnable`

Modify the two Java classes given, such that our threads are created by implementing `Runnable`.

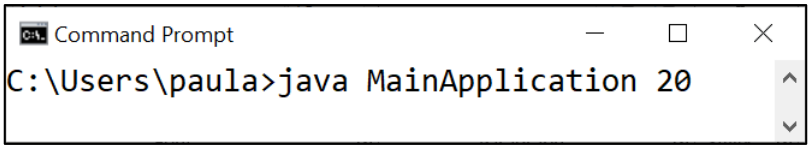
Hint: Look at Oracle's Java tutorial (mentioned in page 1 of this lab sheet) and/or the .zip file under the **Block 3** topic folder **L11: Concurrency and Threads (part 1/2)**.

Task 2: Many threads

Currently, only 2 threads are created in the `MainApplication` class; this is not a flexible way of creating threads.

Modify the code of class `MainApplication`, such that a certain number of threads are created. You can specify the value of the variable (call it `numThreads`) and make it e.g., equal to `10`. This also means that threads must now be created within a loop.

Make your modifications, such that the number of threads to be created is provided as a command line argument when you run the program from the terminal e.g., the command below will create `20` threads:



Hint: It may help to create an array of threads.

Task 3: Add a `Counter` class

Add a `Counter` class that has the following:

1. A `private` variable `count` initially set to zero.
2. A *getter* method that returns the value of `count`.
3. A method with the signature `public void increment(String name)` which increments count by `1`. @param name is the name of the `Incrementer` (i.e., whichever thread called this method). To increase the likelihood of race conditions, use the following implementation:

```
public void increment(String name) {
  int currCount = count; // read
  // introduce a delay between read and write,
  // to "encourage" race conditions
  System.out.println("modify this to print a message that tells
                      us which thread is doing the increment");
  count = currCount + 1; // write
}
```

Task 4: Shared counter object for our threads

Modify the classes `MainApplication` and `Incrementer`, such that:

1. In the `MainApplication` class, a shared `Counter` object is passed in the constructor of our threads.
2. In the `Incrementer` class, modify the constructor accordingly.
3. In the `Incrementer` class, change the `run()` method so that our threads do something mildly useful i.e., increment `count`.
4. In the `Incrementer` class, change the `run()` method so that each thread sleeps for a random amount of time up to 1 second. This will make it more likely to observe races in every run, even for very small loops.

Task 5: `MainApplication` checks if we observed race conditions or not

A. Modify the class `MainApplication`, such that when all threads have finished, it checks whether we observed race conditions or not. Can you figure out how you can do it?

   Hint: What should be the value of `count` if no race conditions were observed?

B. Print a message in class `MainApplication`, to let us know whether races happened in the run, or not.

   Note: Keep in mind that you may need to increase the current duration of "sleep" in the class `MainApplication`, to see your messages printed.

Task 6: Eliminate races

Modify the code, such that races are eliminated.

Hints:

1. You will need to check the lecture materials in the **Block 4** topic in QMplus, to help you achieve this.
2. Have another look at the `Counter` class and the implementation given for its method `increment()` in **Task 3**; what is the ONE thing we usually do to avoid race conditions, when writing methods in multi-threaded applications?

Task 7: Remove sleep from `MainApplication`

The `Thread.sleep()` method makes `MainApplication` wait a bit, until all threads have finished their work. This is not a "nice" way to do it (e.g., if you create many threads and increase `numAdds` significantly, you will also need to increase the value of "sleep").

Can you think of an alternative way to make `MainApplication` wait until all threads have finished before it proceeds, to check if races were observed and print its message?

Hint: Look at the `Thread.join()` method's API and see how you can modify your code to use it. Check the information at https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html.